



Universidad
Rafael Landívar
Identidad Jesuita en Guatemala

Universidad Rafael Landívar

Facultad de Ingeniería

Facultad de Informática y Sistemas

Manejo e Implementación de Archivos

Laboratorio No. 2 - Semana 13

API Secrets y autenticación

Estudiante: Michael Darío Gutiérrez Quel

Carné: 11997

Fecha: 02 de octubre de 2026

1. Objetivo

Practicar el consumo de una API REST autenticada y aplicar una práctica básica de seguridad para evitar que las credenciales queden escritas directamente en el código fuente. La práctica utiliza Postman, una aplicación C# y GitHub.

2. Parte 1 - Autenticación con Postman

2.1 Inicio de sesión

Se creó la solicitud Login con el método POST y el endpoint `https://dummyjson.com/auth/login`. En el cuerpo de la solicitud se enviaron las credenciales de prueba indicadas en el laboratorio.

```
{
  "username": "emilys",
  "password": "emilyspass"
}
```

La API respondió con el código HTTP 200 OK y devolvió un `accessToken`, que posteriormente se utilizó para acceder al recurso protegido del usuario.

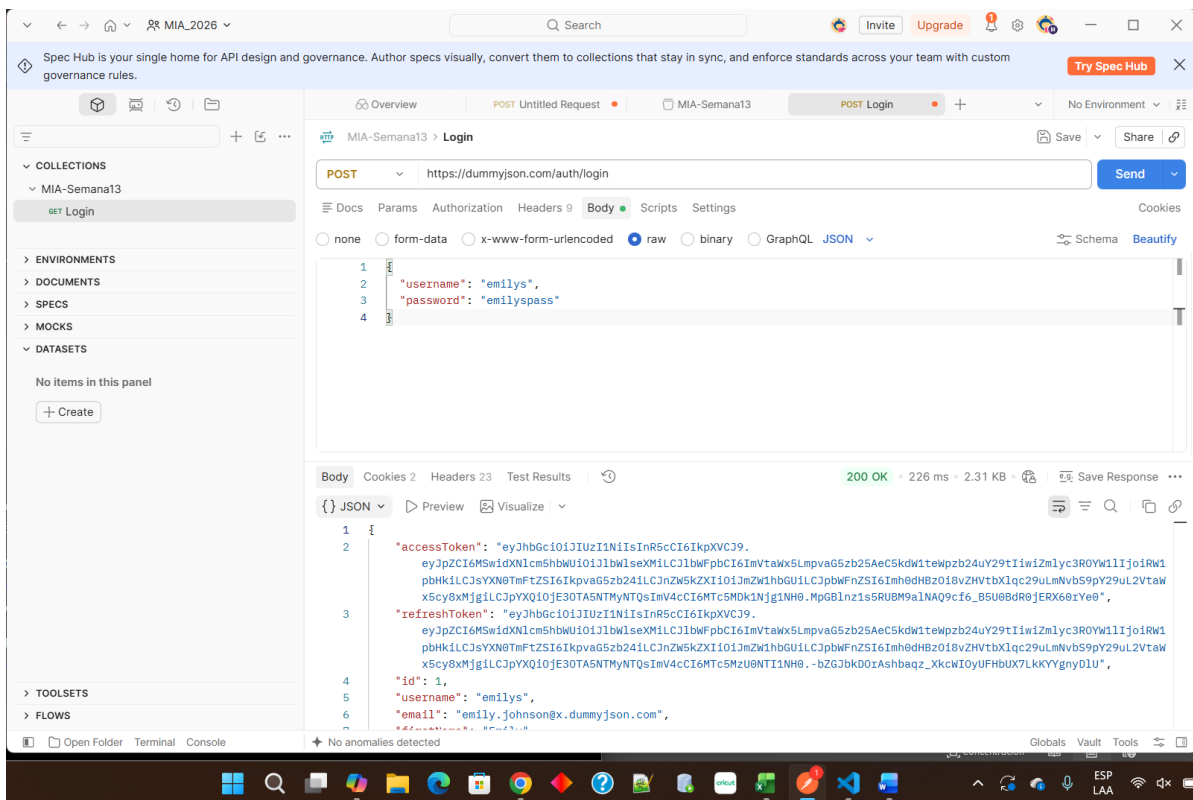


Figura 1. Solicitud de Login ejecutada correctamente y respuesta con accessToken.

2.2 Acceso al usuario autenticado

Se creó una solicitud GET al endpoint `https://dummyjson.com/user/me`. En Authorization se seleccionó Bearer Token y se colocó el accessToken obtenido en el Login. La API respondió con 200 OK y mostró la información del usuario autenticado.

2.3 Prueba sin token

Para comprobar el comportamiento sin autenticación, se realizó la misma solicitud con Authorization configurado como No Auth y sin un token válido. La respuesta fue 401 Unauthorized con el mensaje "Access Token is required".

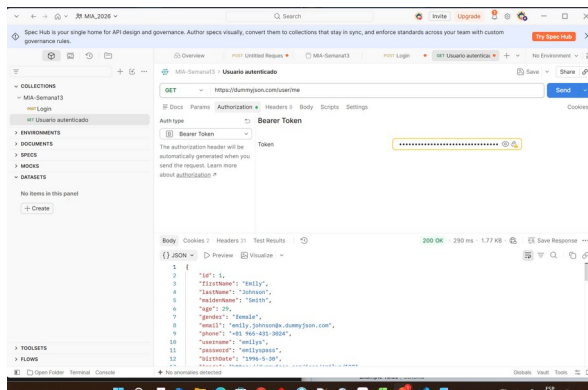


Figura 2. Solicitud autenticada con Bearer Token: 200 OK.

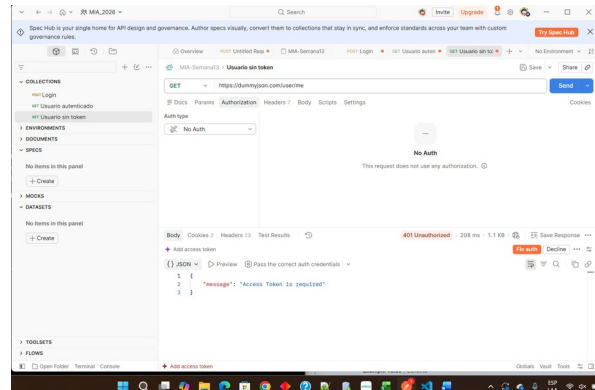


Figura 3. Solicitud sin token: 401 Unauthorized.

2.4 Comparación y función del Bearer Token

Con el Bearer Token la API reconoce la sesión y permite consultar el recurso protegido, por lo que devuelve 200 OK. Sin el token, el servidor rechaza la solicitud y devuelve 401 Unauthorized. El Bearer Token funciona como una credencial temporal que se envía en el encabezado Authorization para demostrar que la solicitud está autenticada y autorizada para acceder al recurso.

3. Parte 2 - Protección de la Connection String de Azure Storage

3.1 Lectura de la credencial desde el entorno

La Connection String no se dejó escrita directamente en Program.cs. El programa obtiene su valor mediante la variable de entorno `AZURE_STORAGE_CONNECTION_STRING`. Para cargar el archivo `.env` durante la ejecución local se agregó el paquete `DotNetEnv`.

```
using DotNetEnv;

Env.Load();

string? connectionString =
    Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
```

3.2 Archivo `.env` y protección con `.gitignore`

En la carpeta principal del proyecto se creó el archivo `.env` con la variable `AZURE_STORAGE_CONNECTION_STRING`. El valor real de la Connection String se mantiene únicamente de forma local. Además, se agregó `.env` al archivo `.gitignore` para impedir que la credencial sea publicada en GitHub.

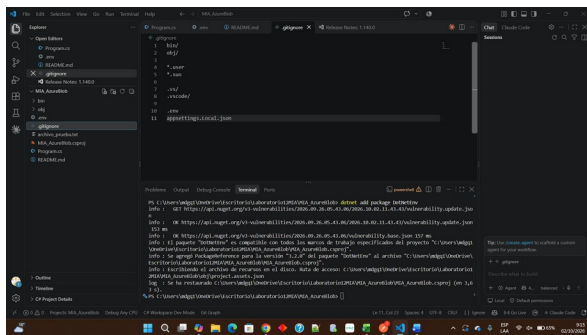


Figura 4. `.gitignore` configurado para excluir el archivo `.env`.

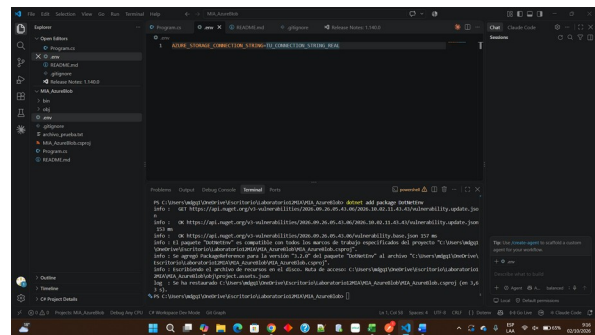


Figura 5. Estructura de la variable utilizada en el archivo `.env`.

3.3 Verificación de funcionamiento

Después de cargar la variable desde .env, el proyecto se compiló y se ejecutó nuevamente. Se utilizó la opción de listar archivos y el programa mostró archivo_prueba.txt almacenado en Azure Blob Storage, confirmando que la aplicación continúa conectándose correctamente sin exponer la Connection String en el código fuente.

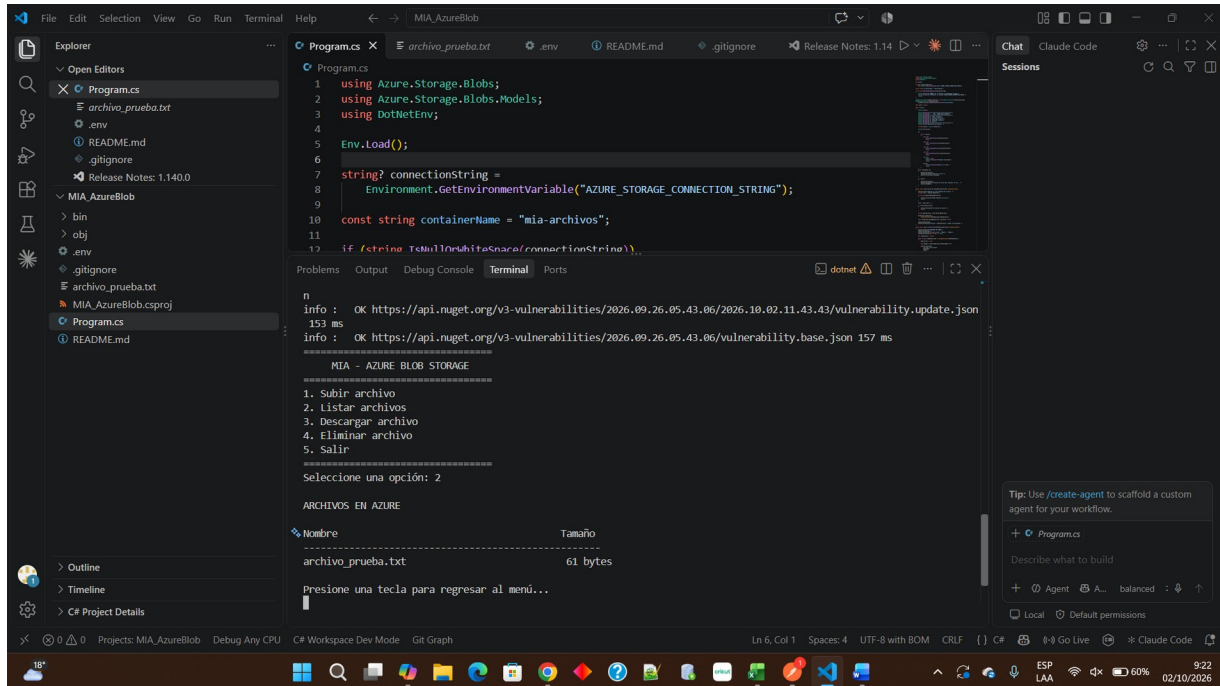


Figura 6. Programa ejecutado correctamente utilizando la variable AZURE_STORAGE_CONNECTION_STRING.

4. Secret del repositorio en GitHub

En el repositorio url-mia-2026 se creó un Repository Secret con el nombre AZURE_STORAGE_CONNECTION_STRING. GitHub conserva el valor cifrado y no muestra la Connection String en la interfaz después de guardarla.

4.1 Evidencia del Repository Secret

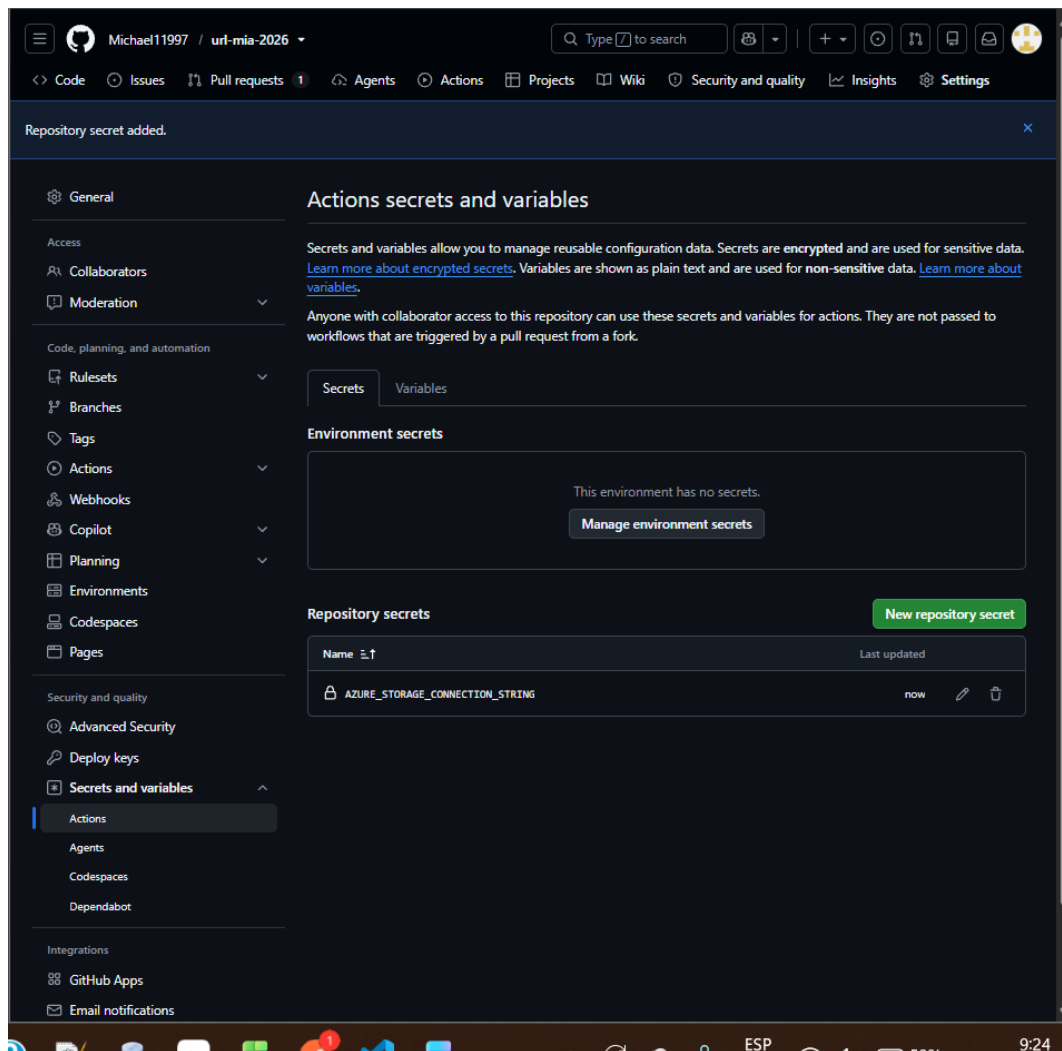


Figura 7. Repository Secret AZURE_STORAGE_CONNECTION_STRING creado en GitHub.

5. Conclusión

La práctica permitió comprobar el uso de autenticación mediante Bearer Token y observar la diferencia entre una solicitud autorizada y una solicitud sin credenciales. También se protegió la Connection String de Azure Storage al moverla fuera del código fuente, cargarla desde un archivo .env excluido del repositorio y almacenarla como un Secret de GitHub. Con estas medidas, el programa mantiene su funcionamiento y se reduce el riesgo de publicar credenciales sensibles en el repositorio.